

PatriMoi

Architecture technique de la version 1.6 — iOS / React Native

Version	1.6 — Juillet 2026	Plateforme	iOS (React Native 0.75.4)
Backend	Supabase (West EU — Ireland)	Statut	Production / TestFlight
Premium	PatriMoi+ — 49 DH/mois	Bundle ID	ma.patrimoi.app

1 Contexte et portée

PatriMoi est une application iOS de **suivi de patrimoine personnel** pour Marocains (Maroc ou diaspora). Elle couvre **8 catégories d'actifs** : liquidités (DH + 18 devises), banque, compte sur carnet, PEA (BVC), compte-titres, or, immobilier, et transport (dépréciation auto des véhicules).

- **Mode démo** — données INIT ~1,5 M DH, aucun compte requis, MMKV local seulement
- **Mode connecté** — compte Supabase, sync cloud, snapshots historiques, PatriMoi+ (49 DH/mois)

Dimension	Valeur
Framework	React Native 0.75.4 — Old Architecture (Paper renderer)
Moteur JS	Hermes 0.13
Backend	Supabase — fwgsdjhavrqrqwmydwx, West EU (Ireland)
Stockage local	MMKV (chiffré AES-128) + Keychain iOS + AsyncStorage (legacy)
Distribution	TestFlight (bêta) → App Store

Évolutions majeures v1.2→v1.6 : page SuiviBudget, export PDF duale (Patrimoine + Budget), module natif RNPDFExport.m (WKWebView), PDF Budget enrichi (santé badge, SVG tendances, positions PEA/CT), restructuration catégories (Transport), score santé patrimoniale.

2 Vue d'ensemble de l'architecture

Architecture **client-heavy**, **BaaS-backed** : toute la logique métier tourne dans le client React Native ; Supabase assure persistance, auth, et cache des données de marché.

Principe directeur	Le client est la source de vérité. Supabase est la sauvegarde cloud, pas un ORM. Toutes les mutations transitent par l'outbox MMKV — Supabase reçoit une copie asynchrone.
---------------------------	--

3 Infrastructure logicielle

3.1 React Native & moteur Hermes

Composant	Version	Rôle
React Native	0.75.4	Framework UI mobile
Hermes	0.13	Moteur JS embarqué (AOT, démarrage rapide)
React	18.3	Librairie UI (hooks, reconciler)
Architecture	Old Architecture	Bridge JS↔Native (JSI non activé — migration v2.0)
Metro	^0.80	Bundler JavaScript (port 8081)
Babel	@react-native/babel-preset	Transformation JSX → CJS

3.2 Configuration deux dépôts

Dépôt	Chemin	Rôle
Source / CI	~/Claude/Projects/PatriMoi	Logique métier, composants, CI GitHub Actions
Build Xcode	~/PatriMoiApp	Pods, node_modules natifs, Xcode project, IPA
Sync	apply_modifs.command	rsync sélectif src/ → PatriMoiApp/ + rm .ts/.tsx conflictuels

Metro shadowing

Metro résout .ts avant .js. `apply_modifs.command` supprime les .ts/.tsx conflictuels avant chaque déploiement.

4 Couche stockage

Niveau	Technologie	Contenu	Sécurité
Niveau 1 (primaire)	MMKV <code>patrimoi_v2</code>	Données patrimoine JSON complètes	AES-128, clé en Keychain
Niveau 1 (clé)	iOS Keychain	Clé AES-128 + JWT Supabase + PISearch	Secure Enclave, device-only
Niveau 2 (legacy)	AsyncStorage	Copie patrimoine (fallback offline)	Non chiffré
Niveau 3 (outbox)	MMKV <code>patrimoi_v2</code>	Queue mutations + verrou <code>updated_at</code>	AES-128 (même store)

Keychain services : `patrimoi.mmkv.key` · `patrimoi.supabase.*` · `patrimoi.pin` — tous `WHEN_UNLOCKED_THIS_DEVICE_ONLY`

Fix critique v1.2 — Scope bug

`mmkvKey` déclaré `const` dans `try` — inaccessible dans `catch`. Corrigé : `let mmkvKey = null` avant le `try`. Sans fix : `key=null` → nouvelle clé → perte totale des données.

5 Composants d'intégration

5.1 Supabase

Service	Usage PatriMoi	Implémentation
Auth (GoTrue)	Inscription, connexion, refresh token	<code>supabase.auth.*</code> via <code>keychainStorage adapter</code>
PostgreSQL + RLS	Stockage JSONB patrimoine, profils, snapshots	<code>supabase.from("patrimoine_data")</code> + RPC
Edge Functions	Proxy APIs marché (BVC, or, devises)	<code>market-data-proxy</code> (Deno)
<code>pg_cron</code>	Mise à jour périodique <code>market_cache</code>	Migration 003 — toutes les 30 min

5.2 Outbox pattern (`syncQueue.js`)

Coalescing last-write-wins · debounce 800ms + worker 5s · max 5 retries backoff exponentiel · verrou optimiste M008 · `stripMarketData()` (R8) avant `upsert`.

5.3 APIs données de marché

Source	Donnée	Transport	Cache
BVC	Cours actions — 70+ tickers	GitHub Actions <code>bvc.yml</code> → JSON cache	MMKV 30min / stale 24h
Or	Prix or DH/gramme	GitHub Actions <code>or.yml</code> → JSON cache	MMKV 30min
BAM	Taux 18 devises → MAD	Fetch direct <code>bkam.ma</code> (XML parsing)	MMKV 30min

BVC — GitHub Actions cron

La BVC n'a pas d'API publique. `bvc.yml` déclenché toutes les 30min scrape les cours et publie un JSON statique. `fetchWithRetry()` le consomme. Évite CORS et Edge Function dédiée.

6 Couche données

6.1 Zustand store

Slice logique	State	Setters
Données patrimoine	data (INIT par défaut)	setData(fn) → sync · setDataRaw(fn) → pas de sync
Navigation	page, sub	setPage(page, sub?)
Auth	user, demoMode, authReady, appReady	setUser, setDemoMode...
UI / Sync	isRefreshing, bvcStatus, discret, history, object	setters correspondants
Budget (v1.6)	budgetOps[] — opérations revenus/dépenses	addBudgetOp, editBudgetOp, deleteBudgetOp

6.2 Schéma de données patrimoine (v1.6)

Catégorie	Clé JSON	Spécificités v1.6
Liquidités	liquidites	18 devises, taux BAM auto-fetch, stripped (R8)
Banque	banque	Comptes courants + épargne séparés de liquidités
Carnet	carnet	Projection 1/3/5 ans (intérêts + versements), taux BAM 2,75%
PEA	pea	PRU pondéré auto, donut chart, fiscal 5 ans, 70+ tickers BVC, stripped (R8)
Compte-titres	ct	Actions + OPCVM P&L, stripped (R8)
Or	or	prixOr recalculé via GitHub Actions JSON cache
Immobilier	immobilier	Valorisation manuelle, loyers, taux rendement locatif
Transport	transport	Dépréciation auto : Voiture 15%/an, Moto 12%/an, Camion 10%/an

7 Couche IHM

Routeur maison Zustand — aucune dépendance React Navigation. state page → AppNavigator.tsx.
NavBar 6 onglets : DAS / ACT / BUD / CON / APR / PAR.

Page	Fichier	Description
Proverbe	PageProverbe.jsx	Splash avec proverbe financier aléatoire (10 proverbes marocains)
Auth	PageAuth.jsx	Login / Signup / Reset password
Onboarding	PageOnboarding.jsx	Première configuration (nom, objectif, date de début)
Dashboard	PageDashboard.jsx	Total patrimoine, sparklines multi-période (1S/1M/3M/6M/1A/MAX), MASI
Actifs	PageActifs.jsx	8 catégories CRUD — SubLiquide (BAM), SubBanque, SubCarnet (proje
SuiviBudget ★NEW	PageSuiviBudget.jsx	Suivi mensuel revenus/dépenses, catégories personnalisées, balance, ta
Conseils	PageConseils.jsx	Recommandations personnalisées (8+ types), score santé 4 dimensions,
À propos	PageAPropos.jsx	Version, licences, contact
Paramètres	PageParams.jsx	Compte, PIN, PatriMoi+ (49 DH/mois), export PDF Patrimoine, export PD

8 Export PDF — v1.6

8.1 Architecture duale

Deux PDF distincts : PDF Patrimoine + PDF Budget. Séparation motivée par la troncature WKWebView (createPDFWithConfiguration tronque au-delà de la hauteur d'une page de rendu si le HTML est trop long avant scroll).

PDF	Contenu	Taille ~
-----	---------	----------

PDF Patrimoine	Total patrimoine, répartition par catégorie, détail actifs, conseils, score santé	1-2 pages A4
PDF Budget	Santé badge, SVG tendances 6 mois, top 3 dépenses, revenus détail, positions PEA, CT, Comparatif mois N-1	3-5 pages A4

8.2 Module natif RNPDFExport.m

Native module iOS exposant generatePDF(htmlString, options, resolve, reject) à JavaScript.

- WKWebView offscreen — chargée avec le HTML généré
- Attente webView:didFinishNavigation: avant capture
- createPDFWithConfiguration — PDF A4 portrait via API iOS native
- Stratégie scroll-per-page — scroll simulé par incréments pour forcer le rendu multi-pages
- Export via UIActivityViewController (Partage iOS)

WKWebView vs react-native-html-to-pdf	WKWebView offre rendu SVG fidèle, CSS variables, webfonts — nécessaire pour les graphiques tendances (SVG inline) et les badges colorés. react-native-html-to-pdf utilise UIWebView déprécié sans SVG fidèle.
--	--

8.3 PDF Budget v1.6 — 9 sections

Section	Contenu	Implémentation
Badge Santé	Score 0/3 (balance≥0 · épargne≥10% · budget≤100%)	scoreScoreContextElg, santéScoreRangeRange)
Résumé mensuel	Revenus / Dépenses / Balance + taux d'épargne	Agrégation budgetOps mois courant
SVG Tendances	Bar chart 6 derniers mois revenus vs dépenses	SVG inline généré en JS — trendSvg, trendMax normalisé
Budget vs Réel	% utilisation budget mensuel défini	budgetPct = totalDep / objectifBudget × 100
Top 3 Dépenses	Trois opérations dépenses les plus élevées du mois	[...depenses].sort((a,b)=>b.montant-a.montant).slice(0,3)
Revenus détail	Liste chronologique opérations revenus du mois	revenuOps triés par date, fmtD(iso) → dd/mm/yyyy
Mois précédent	Comparatif dépenses N vs N-1 + delta	totalDepPrev depuis prevMonthStr
Positions PEA	ticker, quantité, PRU, valeur actuelle, P&L	data.pea avec cours BVC
Positions CT	Actions CT — même structure que PEA	data.ct.actions

Fix date v1.6

fmtD(iso) utilisait iso.split("-") directement → "25T23:00:00.000Z" comme fragment de jour. Fix : const d = String(iso).slice(0,10) avant split → format dd/mm/yyyy garanti dans tous les cas.

9 Conventions de développement

Zero Static Imports : tous les modules natifs via require() dans try/catch, jamais import ES6. Raison : avec Babel CJS (Old Architecture), un import cassé avorte toute la module factory.

Code	Signification
R2	Stockage Keychain (phase 2) — storage.js
R3	Verrou optimiste (M008) — syncQueue.js
R5	Sentry monitoring — sentry.js
R7	Snapshots historiques — auth.js
R8	Strip données de marché avant Supabase — syncQueue.js stripMarketData()
R9	Validation sans Zod — migrations.js validateData()

10 Sécurité

Vecteur	Mesure	Implémentation
Données locales	Chiffrement AES-128 MMKV	patrimoi_v2 — clé en Keychain
Clé	Keychain iOS (Secure Enclave)	WHEN_UNLOCKED_THIS_DEVICE_ONLY
Backup iTunes	Clé absente des backups non chiffrés	Attribut Keychain device-only
PIN	SHA-256 + sel 128 bits	pinHash.js — <hash>:<salt>
JWT Supabase	Keychain (pas AsyncStorage)	keychainStorage adapter
Accès cloud	RLS isolation par user_id	5 politiques RLS sur toutes les tables
Screenshots	Prévention captures	react-native-prevent-screenshot
Mode discret	Masquage montants dans l'UI	State discret → **** à la place des valeurs
Monitoring	Scrubbing PII avant Sentry	beforeSend — montants + emails filtrés

11 Tests

Fichier	Couverture	Type
__tests__/_calc.test.js	14+ tests — calcLiquide, calcBanque, calcPEA, calcCarnet, calcTransp, calcOr, calcImmo, calc	Unitaires
__tests__/_migrations.test.ts	Migration v0→v1, validateData()	Unitaires
__tests__/_phase5.test.js	Migration + mergeHistory + upsertSnapshot + edge cases	Intégration légère
__tests__/_store.test.ts	Zustand store actions (setData, setDataRaw, navigation, discret)	Unitaires

Coverage minimum : **80% sur les lignes** (src/Utils/calc.ts, migrations.ts). Preset jest : react-native, transform TypeScript via babel-jest.

12 CI / CD

Workflow	Déclencheur	Environnement	Durée ~
ci.yml	Push main/develop + PR→main	ubuntu-latest, Node 18 — Jest + tsc --noEmit	2-3 min
release.yml	Tag v*.*.*	macos-15, Node 20, Ruby 3.3 — Fastlane beta (match→2025-pilot)	20-35 min
bvc.yml	Schedule 30min + dispatch	ubuntu-latest — scrape BVC → JSON cache	<1 min
or.yml	Schedule 30min + dispatch	ubuntu-latest — fetch cours or → JSON cache	<1 min

**Personal Team —
Profile expire tous
les 7 jours**

Après expiration : "Provisioning profile expired". Solution : Xcode → Signing & Capabilities → Try Again. Si timeout réseau (réseau corporate bloque Apple servers) : basculer le Mac sur hotspot iPhone, puis Try Again.

13 Argumentaire des choix techniques

React Native

Choix : Partage code iOS/Android futur, logique métier JS, Hermes rapide	Alternative écartée : Swift : plus performant mais pas de partage. Flutter : équipe JS-native, Keychain/MMKV plus limité.
---	--

MMKV

Choix : Synchrone, AES-128 natif, 10× plus rapide qu'AsyncStorage

Alternative écartée : AsyncStorage conservé comme layer legacy pour migration progressive → retiré en v2.0

WKWebView PDF

Choix : Rendu SVG fidèle, CSS variables, createPDFWithConfiguration natif iOS

Alternative écartée : react-native-html-to-pdf : UIWebView déprécié, pas de SVG fidèle — tendances s'affichaient en boîte noire

Supabase

Choix : PostgreSQL + RLS, open-source, West EU (RGPD)

Alternative écartée : Firebase : Firestore NoSQL inadapté à un schéma patrimoine évolutif, RLS moins expressif

Zustand

Choix : API minimaliste (~20 lignes), setData/setDataRaw distinction lisible

Alternative écartée : Redux Toolkit : overhead disproportionné pour un store mono-entité

Navigation maison

Choix : Navigation tabulaire simple, pas de deep linking → routeur Zustand 10 lignes

Alternative écartée : React Navigation : ~800 Ko de bundle, complexité inutile — migration planifiée v2.0 si deep linking requis

Outbox MMKV

Choix : Durabilité mutations : mutations survivent à un crash ou coupure réseau

Alternative écartée : Appel direct Supabase : perte de mutations si crash entre setData et la réponse réseau

SHA-256 pur JS

Choix : Synchrone, zéro dépendance, compatible Hermes 0.13, ~5ms imperceptible pour PIN

Alternative écartée : SubtleCrypto : asynchrone, non disponible tous contextes Hermes 0.13